



UNIVERSITÀ DEGLI STUDI DI PALERMO

DIPARTIMENTO DI INGEGNERIA

CORSO DI LAUREA IN INGEGNERIA INFORMATICA

Documento Progettuale

SPECIFICHE DEL PROGETTO

Divide et Quantifica



Sviluppatore

Giovanni Luca Cusano

Supervisore

Prof.re Luca Cruciata

Anno Accademico

2025 - 2026

Indice Generale

1	Introduzione	2
1.1	Obiettivi del sistema	2
1.2	Definizioni	2
2	Account di Test e Credenziali	2
3	Dettaglio dello Stack Tecnologico	4
3.1	Client (Frontend) – <code>client/package.json</code>	4
3.2	Server (Backend) – <code>server/package.json</code>	4
3.3	Modulo condiviso	5
3.4	Configurazione dell’ambiente (<code>.env</code>)	5
3.5	Inizializzazione del database	5
3.6	Accesso da altri dispositivi (rete locale)	6
4	Attori del Sistema	6
5	Modello Dati e Struttura del Database	7
5.1	Entità principali	7
5.2	Entità di supporto	8
5.3	Relazioni e vincoli	9
5.4	Tabelle indipendenti	9
6	Sistema Proposto	11
6.1	Overview	11
6.2	Requisiti Funzionali	11
6.3	Requisiti Non Funzionali	11
6.3.1	Prestazioni e Architettura	11
6.3.2	Sicurezza e Privacy	11
6.3.3	Usabilità e Design	11
6.3.4	Conformità normativa	12
7	Macro Pagine e Funzionalità Core	12
7.1	Autenticazione & Welcome Page	12
7.2	Dashboard (Infinite Workspace)	12
7.3	Terminale Broker & Nastro di mercato	12
7.4	Classifica (Leaderboard)	13
7.5	Pannello Admin	13
8	Definizioni	13
9	Changelog	13
10	Copyright e diritto d’autore	15

1 Introduzione

1.1 Obiettivi del sistema

“DeQ: Divide et Quantifica” è un’applicazione per l’analisi visiva dei mercati e il trading simulato, pensata per analisti, trader e studenti che vogliono libertà nel comporre i propri strumenti.

L’idea di base è superare le dashboard a griglia fissa: al loro posto c’è uno spazio di lavoro infinito, dove si esplorano i dati in tempo reale, si costruiscono i propri schemi collegando dei nodi e si compra e vende in modo simulato.

In particolare, il sistema punta a:

- **Analisi modulare:** un catalogo di widget (azioni, grafici, aggregatori) da trascinare e collegare per far scorrere e calcolare i dati in tempo reale.
- **Trading simulato realistico:** un Terminale Broker collegato a dati esterni, per provare acquisti e vendite (con commissioni) senza rischiare denaro vero.
- **Classifica e condivisione:** una Leaderboard dove gli utenti pubblicano le proprie strategie, si confrontano sul guadagno o la perdita delle posizioni e si scambiano recensioni.
- **Salvataggio e prestazioni:** salvataggio automatico della canvas sul database e una cache delle quotazioni per ridurre le chiamate esterne.

1.2 Definizioni

Termine	Descrizione
Utente (Trader)	Persona registrata che usa la piattaforma per creare strategie e simulare operazioni.
Workspace (Canvas)	La tela infinita, navigabile con pan e zoom, dove l’utente dispone e collega i nodi per comporre la sua dashboard.
Widget (Nodo)	Modulo autonomo (Azione, Grafico, Orologio, Media...) da mettere sulla canvas. Può fare da sorgente o da destinazione dei dati tramite i collegamenti.
Terminale Broker	La parte per comprare e vendere: listino, carrello, inventario e storico, dove si spende il credito virtuale.
Leaderboard Entry	La scheda che un utente rende pubblica per la classifica. Il punteggio è il guadagno o la perdita delle posizioni, cioè la differenza tra prezzo attuale e prezzo medio d’acquisto.
Provider Dati	Servizio esterno (es. Yahoo Finance) da cui il backend prende quotazioni e serie storiche per grafici e listino.
Token JWT	Il token usato per l’accesso. Grazie a un numero di versione garantisce una sola sessione: un nuovo login disconnette gli altri dispositivi.
Modale	Finestra che appare sopra la schermata e blocca il resto finché non ci si interagisce (es. per confermare un acquisto).

2 Account di Test e Credenziali

Di seguito le credenziali degli account di prova già presenti nel database, per valutare il sistema e i diversi permessi. Per avviare il backend è però necessario prima configurare il proprio file `server/.env` con le chiavi richieste (vedi *Configurazione dell’ambiente* nel capitolo Architettura).

Email	Password	Ruolo	Scopo del Test
admin@gmail.com	adminadmin	Admin	Account principale. Accesso completo al Pannello Admin per la gestione degli utenti e del listino asset.
adminriserva@gmail.com	adminriserva	Utente	Account standard di riserva, utile come profilo secondario di collaudo.
carola@gmail.com	carolauser	Utente	Account standard. Per provare workspace, acquisti nel Broker e pubblicazione in classifica.
emanuele@gmail.com	emanueleuser	Utente	Account standard. Per provare le interazioni tra utenti, come le recensioni sulle schede altrui.
daniela@gmail.com	danielauser	Utente	Account standard. Per riempire la classifica e vederne l'ordinamento con più utenti.
michele@gmail.com	micheleuser	Utente	Account standard. Per provare l'upgrade a Premium e l'acquisto in blocco di un portafoglio dalla classifica.
giuseppe@gmail.com	giuseppeuser	Utente	Account standard. Per provare i collegamenti tra widget e gli aggregatori di calcolo.

3 Dettaglio dello Stack Tecnologico

Il progetto tiene client e server ben separati, ma coerenti tra loro: i due lati condividono i tipi TypeScript e le stesse regole di validazione.

3.1 Client (Frontend) – client/package.json

Libreria	Versione	Scopo
@angular/core (...)	^21.2	Framework SPA (architettura Standalone + Signals)
@ionic/angular	^8.0	Componenti UI ibridi e approccio mobile-first
ionicons	^7.0	Set di icone vettoriali standardizzate
echarts	^6.1	Motore di rendering grafico (candele, linee, aree, volumi)
@jsverse/transloco	^7.5	Gestione dell'internazionalizzazione dinamica (IT/EN)
rxjs	~7.8	Gestione asincrona e programmazione reattiva
zod	^3.25	Validazione rigorosa dei form e dei payload lato client
zone.js / tslib	~0.15 / ^2.8	Runtime Angular e helper TypeScript

Tooling e Dipendenze di Sviluppo (devDependencies):

- **Build/CLI:** @angular/build, @angular/cli, @angular/compiler-cli, @angular/language-service (^21), @ionic/angular-toolkit.
- **Linting:** eslint (^9), i pacchetti @angular-eslint/* e @typescript-eslint/*, oltre a plugin specifici per import, JSDoc e arrow functions.
- **Testing:** karma, jasmine-core, e relativi reporter/launcher per la suite di test.
- **Linguaggio:** typescript ~5.9.

3.2 Server (Backend) – server/package.json

Libreria	Versione	Scopo
express	^4.19	Web framework e gestore dell'infrastruttura API REST
pg	^8.12	Driver nativo per il client PostgreSQL (connessione a Supabase)
jsonwebtoken	^9.0	Generazione e verifica dei token per l'autenticazione stateless
bcryptjs	^2.4	Funzioni di hashing crittografico per la protezione delle password
zod	^3.23	Validazione rigorosa degli input e dei payload in ingresso
nodemailer	^9.0	Modulo di invio email transazionali tramite protocollo SMTP
cors	^2.8	Configurazione e gestione delle policy CORS
dotenv	^16.4	Iniezione e gestione sicura delle variabili d'ambiente

Tooling e Dipendenze di Sviluppo (devDependencies):

- **Esecuzione:** tsx (^4.16) per l'esecuzione nativa e il watch dei file TypeScript in ambiente di sviluppo.
- **Linguaggio e Tipizzazione:** typescript (^5.5) e pacchetti @types/ per Node, Express, pg, JWT, bcryptjs, cors e nodemailer.

3.3 Modulo condiviso

Il progetto ha una cartella **shared/** che fa da ponte tra frontend e backend: non contiene codice eseguibile, solo modelli e interfacce TypeScript usati da entrambi (importabili con l'alias `$shared`). **Zod** è l'unica libreria usata su tutti e due i lati, così le regole di validazione sono scritte una volta sola. Il server usa Express 4 (^4.19), ancora il più compatibile con i middleware disponibili.

3.4 Configurazione dell'ambiente (.env)

Per motivi di sicurezza il repository **non** include il file `server/.env`: contiene chiavi e password che non devono essere condivise pubblicamente. Per far funzionare il backend è quindi necessario **creare un proprio file** `server/.env`, partendo dal modello `.env.example` già presente nel progetto e inserendo le proprie chiavi. Le uniche variabili davvero obbligatorie sono `DATABASE_URL` e `JWT_SECRET`; le altre hanno valori di default. Senza le variabili `MAIL_*` l'applicazione funziona comunque: solo l'invio delle email per il recupero password resta disattivato.

Variabile	Oblig.	A cosa serve e come procurarsela
<code>DATABASE_URL</code>	Sì	Stringa di connessione al database PostgreSQL. Si ottiene creando un progetto gratuito su Supabase (supabase.com) e copiando la connection string dalla sezione <i>Project Settings, Database</i> .
<code>JWT_SECRET</code>	Sì	Chiave segreta per firmare i token di accesso. È una stringa casuale lunga, che ci si genera da soli, ad esempio con <code>openssl rand -hex 32</code> oppure con <code>node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"</code> .
<code>MAIL_USER</code> , <code>MAIL_PASS</code> , <code>MAIL_FROM</code>	No	Credenziali Gmail per l'invio delle email di recupero password. <code>MAIL_USER</code> è l'indirizzo Gmail; <code>MAIL_PASS</code> non è la password normale ma una <i>App Password</i> di 16 caratteri, generata da <i>Account Google, Sicurezza, Password per le app</i> dopo aver attivato la verifica in due passaggi; <code>MAIL_FROM</code> è il mittente mostrato (di norma lo stesso indirizzo).
<code>PORT</code>	No	Porta del server (default 3000).
<code>CORS_ORIGIN</code>	No	Origini ammesse per le richieste; default: tutte quelle locali e di rete locale.
<code>JWT_EXPIRES_IN</code>	No	Durata dei token di accesso (default 7d).
<code>PREMIUM_PRICE</code>	No	Prezzo della licenza Premium (default 10).
<code>QUOTE_CACHE_MINUTES</code>	No	Durata della cache delle quotazioni, in minuti (default 20).
<code>FRONTEND_URL</code>	No	Indirizzo del frontend, usato nei link delle email (default <code>http://localhost:8100</code>).

In pratica: si copia `.env.example` in `.env` e si incollano `DATABASE_URL` e `JWT_SECRET` (più eventualmente le `MAIL_*`).

3.5 Inizializzazione del database

Un database Supabase appena creato è vuoto: le tabelle **non** vengono generate all'avvio del server. La prima volta (o dopo aver cambiato database) va quindi eseguito il comando `npm run migrate` nella cartella `server/`, che applica in ordine gli script SQL in `server/src/db/migrations/` e crea l'intero schema. Le migrazioni sono *idempotenti* (usano `CREATE TABLE IF NOT EXISTS`), quindi si possono rilanciare senza rischi; una di esse inserisce anche alcune azioni di default nel listino. Su un database nuovo non

esistono gli account di prova: il primo utente che si registra diventa automaticamente amministratore. Fatto questo, il server parte con `npm run dev`.

3.6 Accesso da altri dispositivi (rete locale)

Il client viene servito su `localhost`, ma avviandolo con `ionic serve --external` (o `ng serve --host 0.0.0.0`) è raggiungibile anche dagli altri dispositivi collegati alla stessa rete, ad esempio uno smartphone per i test. L'indirizzo IPv4 da usare è quello che il client mostra nella **console all'avvio**, alla voce *Network*: se il computer ha più schede di rete ne compaiono **due**, una per l'*ethernet* e una per il *wireless*; va scelta quella dell'interfaccia connessa alla stessa rete del dispositivo (di norma il wireless). Il backend non richiede alcuna configurazione dell'indirizzo: il frontend ricava automaticamente l'URL dell'API dall'host da cui viene aperto.

4 Attori del Sistema

Gli attori sono i profili, umani o automatici, che interagiscono con il sistema; ognuno ha permessi diversi.

Attore	Tipo	Descrizione	Ruolo e Permessi nel Sistema
Utente Standard (Trader)	Umano	Chi usa la piattaforma per analizzare e simulare.	Accede con una sola sessione attiva. Gestisce i propri workspace, mette e collega i widget, compra e vende nel Broker (con commissione), cura il profilo e partecipa alla classifica (condivisione e recensioni).
Amministratore (Admin)	Umano	Utente con privilegi in più, che supervisiona la piattaforma.	Ha i permessi dell'utente standard più il Pannello Admin: aggiunge o toglie titoli dal listino, elimina account, forza l'aggiornamento dei prezzi e vede i ricavi dell'app.
Scheduler (Sistema)	Automatico	Processo che gira in background sul server Node.js.	Svolge compiti a tempo senza intervento umano: aggiorna prezzi e storico da Yahoo Finance, e gestisce la scadenza dei token di reset.
Provider Dati	Servizio esterno	API di terze parti usata dal backend.	Fornisce i dati reali (quotazioni e serie storiche) per i widget azionari e i grafici.

5 Modello Dati e Struttura del Database

I dati di DeQ sono salvati in un database PostgreSQL ospitato su Supabase. Lo schema tiene separati lo stato finanziario dell'utente, i workspace salvati e lo storico delle transazioni.

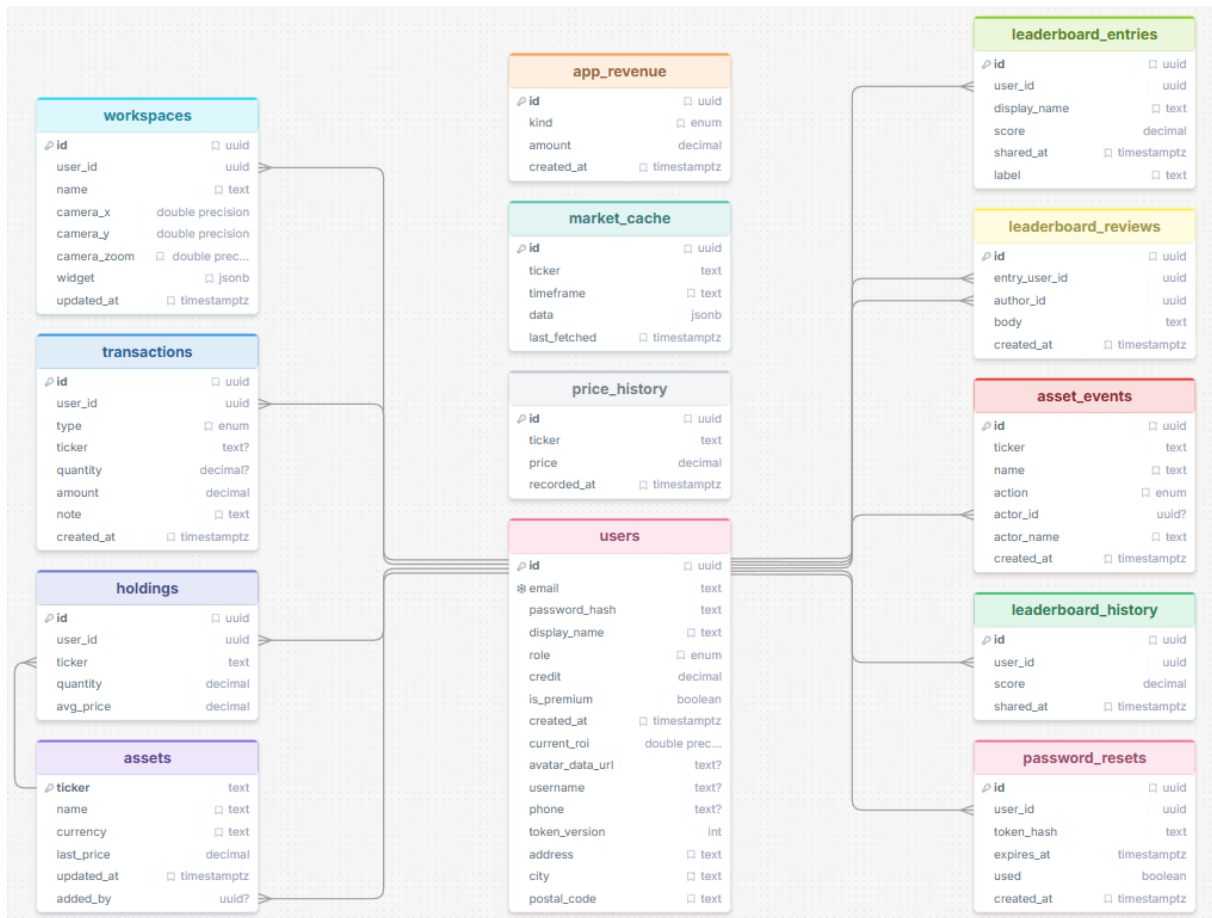


Figure 1: Diagramma Entità-Relazione (ER) del database di sistema generato tramite DrawSQL.

5.1 Entità principali

Sono le tabelle al centro dell'applicazione, quelle che reggono i dati dell'utente e del mercato.

Entità	Descrizione	Attributi Critici	Implicazioni Logiche / Relazioni
User	L'account registrato, con i suoi dati e il suo stato economico.	id, email, credit, is_premium, token_version	Tabella centrale. token_version impedisce i login simultanei. È in relazione 1:N con workspace, transazioni e posizioni.
Asset	Il titolo (azione quotata o personalizzata) che si può seguire e comprare.	ticker, name, last_price, currency	Identificato dal ticker. Alimenta i grafici ed è ciò che si compra nel broker.
Holding	Una posizione: quante quote di un titolo ha un utente.	id, user_id, ticker, quantity, avg_price	Collega User e Asset (relazione M:N). Salva il prezzo medio d'acquisto, usato per il calcolo del guadagno.
Transaction	Il registro (solo aggiunte) di ogni movimento di saldo o stato.	id, user_id, type, amount, created_at	Segna il motivo (acquisto, vendita, ricarica, Premium), utile per lo storico e per l'admin.
Workspace	Una scheda della canvas, con la disposizione dei nodi.	id, user_id, name, camera_zoom, widget (JSONB)	Salva la posizione della vista e l'intera rete di widget e collegamenti in un campo JSON, per il salvataggio automatico.

5.2 Entità di supporto

Tabelle di servizio: classifica, cache dei prezzi, sicurezza e contabilità.

Entità	Descrizione	Funzione Architetturale	Allineamento e Vincoli
Leaderboard Entry	La scheda che un utente rende pubblica per la classifica.	Ordina la classifica in base al guadagno o alla perdita delle posizioni (prezzo attuale meno prezzo medio d'acquisto).	Relazione N:1 con User. Il vincolo unico (<code>user_id</code> , <code>label</code>) permette a un utente di partecipare con più schede.
Leaderboard Review	La recensione che un utente lascia sulla scheda di un altro.	È la parte sociale: commenti e confronto tra utenti.	Due chiavi esterne in CASCADE: verso l'utente recensito (<code>entry_user_id</code>) e verso l'autore (<code>author_id</code>).
Market Cache	Copia temporanea delle risposte di Yahoo Finance.	Riduce le chiamate esterne ed evita di superare i limiti.	Il dato vale circa 20 minuti (in base a <code>last_fetched</code>). Indicizzata per ticker e timeframe.
Price History	Storico dei prezzi registrati nel tempo dal sistema.	Serve al nastro di mercato e al broker per calcolare le variazioni.	Tabella a sé (solo aggiunte). Si aggancia ai titoli solo per ticker, così lo storico resta anche se un titolo viene tolto.
Password Reset	Il record temporaneo per il recupero password.	Conserva l'hash del token e la scadenza (1 ora) per validare il link via email.	Relazione N:1 con User, con un flag <code>used</code> per non riutilizzare lo stesso token.
App Revenue	Registro dei ricavi (fitizi) dell'app.	Somma gli incassi da Premium (€10) e dalle commissioni (1%) per i grafici dell'admin.	Tabella isolata (nessuna chiave esterna), così i dati contabili restano anche se un utente viene eliminato.

5.3 Relazioni e vincoli

La coerenza dei dati è affidata direttamente ai vincoli del database:

- **Cancellazione a cascata (ON DELETE CASCADE):** eliminando un utente si cancellano anche i suoi dati collegati: posizioni (`holdings`), transazioni, workspace e token di recupero password.
- **Riferimento azzerato (ON DELETE SET NULL):** le colonne che indicano chi ha aggiunto un titolo (`added_by` in `assets`) o chi ha fatto un'azione (`actor_id` in `asset_events`) vengono messe a NULL se quell'admin viene eliminato, così lo storico resta.
- **Vincoli di unicità (UNIQUE):** la coppia (`user_id`, `ticker`) in `holdings` evita di spezzare una stessa azione in più righe; la coppia (`user_id`, `label`) in `leaderboard_entries` permette a un utente di avere più schede in classifica, ma senza duplicati.

5.4 Tabelle indipendenti

Alcune tabelle sono volutamente slegate dalle chiavi esterne, così restano valide anche se il listino cambia:

- **Storico e cache dei prezzi:** `market_cache` e `price_history` si agganciano ai titoli solo tramite il `ticker` (testo). Se un'azione viene tolta dal listino, lo storico e la cache restano comunque disponibili.
- **Contabilità:** `app_revenue` (ricavi da commissioni e Premium) è una tabella a sé: i conti dell'app non cambiano anche se l'utente che li ha generati viene eliminato.

6 Sistema Proposto

6.1 Overview

DeQ (Divide et Quantifica) è un'applicazione web per l'analisi dei mercati e il trading simulato. A differenza delle dashboard classiche a griglia fissa, offre una tela infinita su cui l'utente dispone e collega liberamente dei moduli, i *widget*: azioni, grafici, nodi di calcolo.

Accanto all'analisi c'è un **Terminale Broker** simulato, dove si comprano e vendono asset con un credito virtuale e una commissione dell'1% su ogni operazione.

L'app funziona su desktop e mobile ed è pensata per trader, analisti e studenti. Una **Classifica (Leader-board)** permette di condividere le proprie schede e confrontarsi con gli altri sul guadagno o la perdita delle posizioni.

6.2 Requisiti Funzionali

Il sistema deve permettere all'utente di seguire tutta la propria strategia: idearla, provarla con operazioni simulate e infine condividerla.

Ci sono due tipi di utente: l'**Utente Standard (Trader)** e l'**Amministratore**. Chi non è registrato non può usare l'app: per accedere agli strumenti e al listino bisogna registrarsi.

L'**Utente** deve poter:

- creare e gestire più **workspace**, salvati in automatico e quindi disponibili su qualsiasi dispositivo;
- mettere **widget** sulla canvas (sorgenti dati, grafici, nodi di calcolo) e collegarli tra loro;
- **comprare e vendere** asset in modo simulato, su un listino aggiornato;
- **pubblicare le proprie schede in classifica** e lasciare recensioni agli altri utenti.

All'**Amministratore** serve un **Pannello di Controllo** per gestire gli utenti, aggiungere o togliere titoli dal listino e vedere i ricavi (fittizi) dell'app.

6.3 Requisiti Non Funzionali

6.3.1 Prestazioni e Architettura

- **Salvataggio automatico**: ogni modifica al workspace viene salvata in background, senza interrompere il lavoro. Lo stato della canvas è convertito in JSON e memorizzato nel database.
- **Cache delle quotazioni**: per non superare i limiti di richieste di Yahoo Finance, il backend conserva per un po' le quotazioni già scaricate e le riusa, riducendo le chiamate esterne.

6.3.2 Sicurezza e Privacy

- **Sessione singola (JWT)**: l'accesso usa token JWT. A ogni nuovo login il token precedente viene invalidato (*token_version*), così l'account non resta aperto su più dispositivi.
- **Recupero password sicuro**: il reset avviene tramite un token inviato via email, valido un'ora e utilizzabile una sola volta.

6.3.3 Usabilità e Design

- **Navigazione fluida**: l'interfaccia gestisce mouse, touch e pennino (*Pointer Events*) per pan, zoom e scorrimento senza scatti.

- **Errori leggibili e localizzati:** il server risponde con codici d'errore stabili, che il client traduce nella lingua scelta (IT/EN); così, se i dati di mercato non sono disponibili o si inserisce un valore sbagliato, l'utente vede un messaggio chiaro invece di un errore tecnico.

6.3.4 Conformità normativa

- **GDPR (Regolamento UE 2016/679):** eliminando un account vengono cancellati a cascata (CASCADE) tutti i suoi dati: profilo, portafoglio, transazioni e workspace ("diritto all'oblio").

7 Macro Pagine e Funzionalità Core

7.1 Autenticazione & Welcome Page

È la pagina d'accesso. Per registrarsi bastano nome, username, email e password (con conferma); indirizzo e telefono si aggiungono dopo, dal profilo. C'è anche il recupero della password via email e, dalla stessa schermata, il recupero dell'email collegata all'account.

7.2 Dashboard (Infinite Workspace)

Il cuore di DeQ: una tela infinita dove l'utente costruisce la propria analisi, con un'interfaccia scura in stile neon.

- **Canvas infinito:** pan, zoom con la rotella e pinch-to-zoom fluidi, su uno sfondo a griglia animata.
- **Schede (tab):** più schede per separare le strategie, con salvataggio automatico in background. Il numero di schede aperte insieme dipende dalla licenza Premium.
- **Annulla (Undo):** l'app tiene la cronologia delle azioni; con **Ctrl+Z** (o **Cmd+Z**) si annulla l'ultima operazione e si recupera un widget o un collegamento cancellato per sbaglio.
- **Catalogo widget:** un pannello da cui trascinare i moduli sulla canvas. I widget adattano il testo al livello di zoom e hanno un look *glassmorphism*. Comprende:
 - **Azione:** segue un singolo titolo (quotato o personalizzato) con dati reali.
 - **Grafici (ECharts):** candele, linea, area, volumi e torta.
 - **Aggregatori:** nodi di calcolo (netto delle posizioni, prezzo medio d'acquisto).
 - **Utility:** inventario, orologio e blocchi di testo.
- **Collegamenti:** si clicca per collegare una sorgente a un consumatore; i dati passano lungo linee animate. Un controllo evita i cicli infiniti.
- **Inspector:** un pannello laterale che cambia in base al widget selezionato. Da qui si scelgono i *timeframe* (da 1 giorno a 5 anni), si forzano le connessioni e si cambiano i colori.

7.3 Terminale Broker & Nastro di mercato

Il punto in cui l'analisi diventa azione (simulata).

- **Terminale e carrello:** listino aggiornato, ricerca dei titoli e checkout per acquisti multipli col credito virtuale. Il saldo si aggiorna da solo, l'operazione viene registrata e si applica l'1% di commissione.
- **Inventario e storico:** schede per rivedere gli ordini fatti (con mini-grafici) e operare sulle posizioni. Da ogni riga dell'inventario si può comprare, vendere o aggiungere al carrello al volo; se manca il credito appare subito un avviso.

- **Nastro di mercato:** la striscia in basso mostra le variazioni percentuali in tempo reale; cliccando un titolo lo si aggiunge al carrello.

7.4 Classifica (Leaderboard)

Serve a condividere le strategie e confrontarsi con gli altri. L'utente pubblica una o più schede; la classifica le ordina in base al guadagno o alla perdita delle posizioni, cioè la differenza tra il prezzo attuale e il prezzo medio d'acquisto, senza contare il patrimonio totale. Ogni scheda ha un'etichetta e può essere ritirata quando si vuole; non si può pubblicare due volte la stessa. Si può personalizzare il profilo con un avatar, lasciare recensioni agli altri e comprare in blocco il portafoglio di un altro utente (sempre simulato). Dal profilo, con la licenza Premium e tutti i campi compilati, si sblocca anche l'accessorio del progetto, il braccialetto DeQ.

7.5 Pannello Admin

Il pannello riservato all'amministratore.

- **Gestione piattaforma:** elenca gli utenti registrati e permette di eliminarli, aggiungere o togliere titoli dal listino (collegato a Yahoo Finance) e riempirlo in fretta con titoli scelti a caso da un ampio elenco. Mostra anche lo stato dell'aggiornamento automatico dei prezzi, con conto alla rovescia e possibilità di forzarlo.
- **Finanze:** riepilogo dei ricavi simulati, tra licenze Premium e commissioni sulle operazioni.

8 Definizioni

Termine	Descrizione
Utente (Trader)	Persona registrata che usa la piattaforma per creare strategie e simulare operazioni.
Workspace (Canvas)	La tela infinita, navigabile con pan e zoom, dove l'utente dispone e collega i nodi per comporre la sua dashboard.
Widget (Nodo)	Modulo autonomo (Azione, Grafico, Orologio, Media...) da mettere sulla canvas. Può fare da sorgente o da destinazione dei dati tramite i collegamenti.
Terminale Broker	La parte per comprare e vendere: listino, carrello, inventario e storico, dove si spende il credito virtuale.
Leaderboard Entry	La scheda che un utente rende pubblica per la classifica. Il punteggio è il guadagno o la perdita delle posizioni, cioè la differenza tra prezzo attuale e prezzo medio d'acquisto.
Provider Dati	Servizio esterno (es. Yahoo Finance) da cui il backend prende quotazioni e serie storiche per grafici e listino.
Token JWT	Il token usato per l'accesso. Grazie a un numero di versione garantisce una sola sessione: un nuovo login disconnette gli altri dispositivi.
Modale	Finestra che appare sopra la schermata e blocca il resto finché non ci si interagisce (es. per confermare un acquisto).

9 Changelog

Gli aggiornamenti più recenti, tra un po' di pulizia e qualche aggiunta:

- Il trascinamento e il ridimensionamento dei widget sono diventati direttive riusabili, al posto di codice ripetuto in ogni componente.

- Inventario e connection hub ora si ridimensionano davvero (i contenuti scalano con la dimensione) e l'orologio non va più a capo.
- È stato tolto un po' di codice inutilizzato e uniformati alcuni nomi.
- I messaggi di errore del server sono ora codici stabili tradotti dal client, così seguono la lingua scelta.
- È stato aggiunto il braccialetto DeQ nel profilo, sbloccabile con il Premium e il profilo completo.

10 Copyright e diritto d'autore

Il presente documento, inclusi i testi, i diagrammi UML, le specifiche dei casi d'uso e le scelte architettoniche in esso contenute, è di proprietà esclusiva degli autori. È vietata la riproduzione, la duplicazione, la distribuzione, anche parziale, o la rielaborazione non autorizzata con qualsiasi mezzo, elettronico o meccanico, senza il preventivo consenso scritto degli autori.